

A condition is an expression that produces True or False

```
age = 16

# A comparison is a value
can_vote = age >= 18

print(can_vote)    # False
print(type(can_vote)) # <class 'bool'>
```

Read it out loud: “is age greater than or equal to 18?”
The answer is a `bool` — a real value you can store, print, and check the type of.

The six comparison operators

OPERATOR	ASKS
<code>==</code>	Are these equal?
<code>!=</code>	Are these different?
<code><</code>	Is the left smaller?
<code><=</code>	Smaller or the same?
<code>></code>	Is the left bigger?
<code>>=</code>	Bigger or the same?

There is no `===` in Python. `==` is the only equality test you need.

One equals sign assigns. Two compare. `price = 8` puts 8 into `price`. `price == 8` asks whether it is already 8. Writing `if price = 8:` is a `SyntaxError` — Python stops you before the program runs.

Strings compare too — alphabetically

```
print("apple" < "banana")    # True
print("Zoe" < "adam")        # True
print("cat" == "Cat")        # False
```

Two things to remember:

- Comparison is **case sensitive** — `"cat"` and `"Cat"` are different strings.
- Every capital letter sorts **before** every lowercase letter, which is why `"Zoe" < "adam"` is True.

Compare numbers as numbers. `input()` always hands you a string. `"9" > "17"` is True, because strings compare character by character. Convert with `int()` first, and `9 > 17` is False, which is what you meant.

The if statement

```
if temperature > 80:
    print("It's hot!")
    print("Drink water")

print("Have a good day")
```

The two indented lines run only when the condition is True. The last line is not indented, so it runs every time.

Three required parts

<code>if</code>	The keyword and something that evaluates to True or False.
<code><condition></code>	
<code>:</code>	A colon ends the line. Not optional.
body	Indented 4 spaces. The indentation is the block — there are no braces to fall back on.

The block ends when the indentation ends.

Choosing between two paths — if / else

```
age = 16

if age >= 18:
    print("You can vote")
else:
    print("Not yet")
```

Exactly one branch runs. Never both. Never neither. `else` takes a colon but no condition — it catches everything the `if` did not.

Truthiness — when a value acts like True or False

```
print(bool(1))      # True
print(bool(0))      # False
print(bool("hello")) # True
print(bool(""))     # False
```

The falsy values

0	The number zero
""	The empty string
False	The boolean itself
None	The "no value" value

Everything else is **truthy**.

Why it is useful

```
name = input("Your name: ")

if name:
    print(f"Hi {name}!")
else:
    print("You didn't enter anything")
```

`if name:` means "if name is not empty." It does the same job as `if name != "":` and reads better.

Watch out for zero. A count of `0` is real data, but it is falsy. `if count:` is False when the count is zero. If zero matters, write `if count > 0:` or `if count != 0:` and say what you mean.

The pattern you will use all unit

```
# 1. Branch to DECIDE a value
if weight > 10:
    shipping = 12
else:
    shipping = 5

# 2. USE the value afterward, outside the branches
print(f"Shipping: ${shipping}")
```

Decide inside the branches. Use the result after them.

Putting the `print` inside both branches means writing it twice. When the wording changes, you have to remember to change it in both places — and eventually you will not.

Common mistakes to avoid

MISTAKE	WHAT GOES WRONG
<code>if score = 70:</code>	<code>SyntaxError</code> . Use <code>==</code> to compare.
<code>if score > 70</code>	Missing colon. <code>SyntaxError</code> pointing at the next line.

Body not indented	<code>IndentationError: expected an indented block</code> .
Mixing tabs and spaces	<code>TabError</code> . Looks fine on screen, still breaks. Use spaces.
<code>else age < 18:</code>	<code>else</code> never takes a condition. Just <code>else:</code> .
True / False spelled lowercase	<code>NameError</code> . Python capitalizes both.
Comparing <code>input()</code> without converting	Compares text, not numbers. <code>"9" > "17"</code> is True.

Today's syntax — quick reference

GOAL	CODE
Run code when something is true	<code>if age >= 18:</code>
Do something else otherwise	<code>else:</code>
Check equality	<code>if answer == "yes":</code>
Check inequality	<code>if answer != "yes":</code>
Check that text is not empty	<code>if name:</code>
Check a number is not zero	<code>if count > 0:</code>
See if a value is truthy	<code>print(bool(value))</code>
Compare a number from input	<code>age = int(input("Age: "))</code>

Before you submit — check every condition

- Does every `if` and `else` line end with a colon?
- Is every block indented 4 spaces, with spaces and not tabs?
- Did you use `==` to compare, not `=`?
- Did you convert with `int()` before comparing numbers?
- Are `True` and `False` capitalized?
- Run it twice — once so the `if` fires, once so the `else` does. A branch you never tested is a branch you never checked.

What is next. Right now you can split a program into two paths. That is enough for yes-or-no questions, but not for three or more options — try adding a third price tier using only `if` and `else` and you will end up burying one `if` inside another. Next lesson introduces `elif`, which handles as many branches as you need without the nesting.

Keep this sheet. Every program from here on decides something. The colon, the indentation, and the difference between `=` and `==` do not change — they just get used inside loops instead of on their own.